

OrbiQuest

Documentație tehnică a codului — structură generală și funcțiile fiecărei pagini

Escape room educațional de astrofizică/astronomie, realizat ca proiect HTML/CSS/JS static, fără build step și fără dependențe externe la runtime — rulează 100% offline prin simpla deschidere a fișierului **index.html** într-un browser.

Fișiere sursă: index.html · modul01–04.html · gameover.html · shared.js · style.css (+ fișiere audio .mp3 folosite ca efecte sonore/ambientale).

Structura generală a proiectului

Jocul e o serie de **pagini HTML separate** (nu un Single Page App) — fiecare cameră/modul e propriul fișier .html, navigarea între ele fiind o încărcare reală de pagină nouă (`location.href = ...`), nu rutare JS. Toate paginile includ același fișier **shared.js**, care conține toată logica reutilizată: generarea deterministă a parametrilor puzzle-urilor, cronometrul de oxigen, tranzițiile animate de tip “ușă de airlock” și motorul audio. Stilul vizual (tema sci-fi HUD, fonturi, culori, componente) e centralizat în **style.css**, cu mici suprascrieri de culoare per-pagină pentru tematica fiecărui modul (cyan / mov / roșu).

Cum circulă starea jocului între pagini

Nu există server sau bază de date — starea jocului călătorește exclusiv prin **parametrii din URL**, transmiși de la o pagină la alta prin link-urile de tranziție:

?seed=NNNNN	generat aleator o singură dată, la apăsarea butonului Start; determină TOȚI parametrii puzzle-urilor, în mod reproductibil (același seed → aceleași valori pe orice pagină).
&c1;= / &c2;= / &c3;=	cifrele de cod obținute până acum, adăugate pe rând, pe măsură ce jucătorul rezolvă fiecare modul.
&t;=SS	secunde rămase din rezerva de oxigen; cronometrul continuă de unde a rămas la fiecare schimbare de pagină.

Funcția `buildParams(seed)` din `shared.js` recalculează, identic pe fiecare pagină, toate valorile fizice ale puzzle-urilor pornind de la același seed, folosind un generator de numere pseudo-aleatoare determinist (`makeRNG`, algoritmul `mulberry32`) — astfel jocul e diferit la fiecare partidă, dar perfect reproductibil în cadrul aceleiași partide.

Audio

Un motor audio unic (obiectul `SFX` din `shared.js`) combină două surse de sunet: efecte sintetizate direct în JS prin **Web Audio API** (clic de tastare, teme ambientale generate din oscilatoare, sunete de reușită/eșec) și **redarea unor fișiere .mp3 locale** (uși, whoosh-uri, ambientale descărcate), incluse fizic în folderul proiectului — nimic nu se încarcă din rețea, deci restricția “rulează offline” rămâne respectată. Un buton ☐ (montat automat pe fiecare pagină prin `mountAudioToggle()`) permite oprirea totală a sunetului; preferința se ține minte în `localStorage`.

Fluxul jocului

`index.html` (intro + landing) → `modul01.html` (mecanică orbitală) → `modul02.html` (efect Doppler) → `modul03.html` (paralaxă) → `modul04.html` (terminal final, introducerea codului) → ecran de victorie. Dacă oxigenul ajunge la zero pe oricare pagină, jucătorul e redirecționat spre `gameover.html`.

shared.js

Modulul comun, inclus de toate paginile — nu se afișează nimic din el direct, dar conține aproape toată logica reutilizabilă a jocului.

makeRNG(seed)	Generator de numere pseudo-aleatoare determinist (mulberry32). Din același seed produce mereu aceeași secvență de numere.
qs()	Scurtătură — întoarce URLSearchParams pentru query string-ul paginii curente.
getSeed()	Citește parametrul ?seed= din URL.
getCodes()	Citește cifrele de cod obținute până acum (c1, c2, c3) din URL.
getTimeLeft()	Citește secunde rămase din oxigen (?t=); implicit 600s (10 min) dacă lipsește.
nextURL(page, extra)	Construiește URL-ul paginii următoare, păstrând toți parametrii curenți și adăugând valori noi (ex. cifra tocmai obținută).
buildParams(seed)	Calculează determinist TOȚI parametrii celor 3 puzzle-uri (rază orbitală țință, deplasare spectrală, paralaxă) și cifrele de cod corespunzătoare, pornind doar din seed.
fmtTime(s)	Formatează secunde ca M:SS.
startOxygen(displayEl, onZero)	Pornește cronometrul de oxigen (interval de 1s), actualizează afișajul și apelează onZero() (redirect spre gameover.html) la epuizare.
renderCodeDisplay(el, codes)	Afișează cifrele de cod obținute până acum (sau “_” pentru cele lipsă) într-un element din antet, ca jucătorul să nu trebuiască să le rețină singur.
renderRail(el, step)	Randează bara de progres cu cele 4 noduri (module 1-3 + Core), marcând care sunt rezolvate/active/blocate.
ensureAirlock()	Creează (o singură dată) elementul DOM al ușilor de airlock, dacă nu există deja pe pagină.
playTransition(kind, label, sub, destURL)	Pornește animația + sunetul de închidere a ușilor, apoi navighează la destURL.
openAirlockIn()	Apelată la încărcarea fiecărei pagini de joc — pornește animația + sunetul de deschidere a ușilor, dezvăluind camera.
SFX.ensure()	Creează (lazy) AudioContext-ul și nodul de câștig principal (master), dacă nu există deja.
SFX.unlock()	Deblochează redarea audio la prima interacțiune a utilizatorului (politica anti-autoplay a browserelor).
SFX.setMuted(m) / isMuted()	Activează/dezactivează sunetul global; ține minte preferința în localStorage.
SFX.tick()	Sunet sintetizat scurt (clic) — folosit la fiecare literă din intro-ul typewriter.
SFX.startAmbient(name) / stopAmbient()	Pornește/oprește o temă ambientală sintetizată (pad de oscilatoare + LFO), unică per modul (teme disponibile: m1, core).
SFX.chime(ok)	Redă un mic arpeggiu sintetizat de reușită (ok=true) sau eșec (ok=false).
SFX.playFile(src, vol, loop)	Redă un fișier audio local (.mp3), cu volum și loop opționale; folosit pentru toate efectele/ambientalele bazate pe fișier.
mountAudioToggle()	Creează și inserează butonul flotant ☐ /☐ (o singură dată per pagină).
\$(s)	Scurtătură pentru document.querySelector(s).

index.html

Pagina de start (landing page): intro typewriter, jurnalul de bord, cardurile celor 3 module și butonul de pornire a misiunii.

(IIFE) boot intro	Bloc auto-executat care rulează la încărcarea paginii: scrie efectul typewriter linie cu linie peste ecranul de “boot”, apoi îl ascunde (animație slide-left) pentru a dezvălui pagina de start.
finish()	Încheie intro-ul: ascunde ecranul typewriter, redă sunetul de whoosh, apoi pornește ambientala de fundal a paginii, în loop.
typeStep()	Recursivă (prin setTimeout) — scrie efectul typewriter caracter cu caracter, linie cu linie, cu sunet de tastare pe fiecare literă.
resize()	Ajustează dimensiunea canvas-ului decorativ din hero la dimensiunea reală a elementului.
draw(now)	Buclo de animație (requestAnimationFrame) a graficii din hero: câmp de stele, rețea de noduri conectate, cub 3D rotativ.
startBtn.onclick	Generează un seed nou aleator și pornește tranziția (uși + navigare) spre modul01.html, cu oxigenul inițial de 10 minute.

modul01.html

Modulul 1 — Laboratorul de Astrofizică: puzzle de mecanică orbitală ($v = \sqrt{GM/r}$). Jucătorul reglează raza orbitei și viteza de injecție a unei sonde.

drawScene1()	Randează scena canvas: planeta, orbita-țintă (linie punctată) și traiectoria simulată a sondei.
launch1()	Lansează simularea: pornește integrarea fizică (physicsTick) și randarea (render) în paralel.
physicsTick() (nested)	Avansează simularea gravitațională pas cu pas, pe bază de timp real (setInterval, independent de fps), și verifică dacă orbita rezultată e stabilă (excentricitate mică) și pe raza corectă.
render() (nested)	Bucula de randare separată de fizică (requestAnimationFrame), ca desenul să nu încetinească verdictul simulării.
r1El.oninput / v1El.oninput	Actualizează afișajele sliderelor de rază/viteză și resetează starea de verdict la fiecare mișcare.
#launch1 / #check1 (click)	Butoane: relansează simularea, respectiv extrage cifra de cod dacă orbita e stabilă și pe raza corectă (redă chime de succes, actualizează codul afișat, apoi tranziție spre modul02.html).

modul02.html

Modulul 2 — Observatorul Astronomic: puzzle de efect Doppler (deplasare spre roșu). Jucătorul aliniază un cursor pe linia spectrală observată și calculează viteza galaxiei.

<code>w1Color(w1)</code>	Convertește o lungime de undă (nm) în culoarea RGB aproximativă vizibilă, pentru a desena spectrul curcubeu.
<code>drawSpectrum()</code>	Randează banda spectrală colorată, linia de laborator (656.3nm), linia observată (deplasată) și cursorul mobil al jucătorului.
<code>w2.oninput</code>	Actualizează afișajul lungimii de undă selectate și redesenează spectrul pe măsură ce cursorul e mișcat.
<code>#check2 (click)</code>	Validează dacă cursorul e aliniat pe linia observată ȘI dacă viteza introdusă de jucător e corectă (calculată din redshift); dacă da, extrage cifra de cod și face tranziția spre modul03.html.

modul03.html

Modulul 3 — Reactorul Cuantic: puzzle de paralaxă trigonometrică. Jucătorul comută între două poziții de observare și calculează distanța până la o stea.

drawParallax()	Randază scena: Pământul pe orbită, poziția curentă de observare (Ianuarie/Iulie), steaua țintă și linia de vizare care arată deplasarea aparentă pe fundalul stelar.
#toggle3 (click)	Comută poziția de observare între Ianuarie și Iulie și redesenează scena.
#check3 (click)	Validează distanța introdusă de jucător (în parseci, calculată din unghiul de paralaxă); dacă e corectă, extrage cifra de cod și face tranziția spre modul04.html (terminalul final).

modul04.html

Terminalul final ("Core"): jucătorul introduce, pe un keypad, codul de 3 cifre colectat din module, pentru a debloca repornirea navei.

renderLED()	Actualizează cele 3 afișaje LED ale codului introdus (cifră/underscore, cursor pe poziția activă).
#keypad (click)	Gestionează apăsarea tastelor virtuale: adaugă cifre, CLR golește codul introdus, ENTER trimite spre validare (submit).
keydown (document)	Permite introducerea codului și de la tastatura fizică (cifre, Backspace, Enter).
clearFb()	Golește mesajul de feedback al terminalului.
submit()	Compară codul introdus cu cel real (din c1+c2+c3, luate din URL); dacă e corect declanșează secvența de succes, altfel afișează eroare + sunet de eșec.
drawCore(now)	Bucula de animație a reactorului pulsatoriu din bara superioară (canvas), cât timp terminalul așteaptă codul.
showAccessGranted()	Oprește tema ambientală, redă chime de succes, afișează pop-up-ul "Acces garantat" cu particule, apoi declanșează plecarea navei.
launchDeparture()	Pornește secvența de plecare: sunet de whoosh urmat de sunetul de decolare, animație canvas de "salt în viteză" (warp), apoi afișează ecranul de victorie.
showWin()	Ascunde terminalul și afișează ecranul final de victorie, cu timpul de oxigen rămas la finalul jocului.
#restart (click)	Reîntoarce jucătorul la index.html pentru o partidă nouă.

gameover.html

Ecranul de eșec — afișat automat când rezerva de oxigen ajunge la zero, indiferent de pagina pe care se afla jucătorul.

Nu definește funcții proprii — doar apelează direct `SFX.ensure()` și `SFX.chime(false)` din `shared.js`, pentru sunetul de eșec la afișarea paginii.

style.css

Nu conține funcții (e CSS), dar centralizează întregul sistem vizual: fontul Orbitron înglobat (base64, pentru offline), variabilele de temă (culori, accent, panouri glassmorphism), câmpul de stele animat de fundal, componentele reutilizate pe toate paginile (antet cu statistici, bara de progres “rail”, panourile “room”, dial-urile de slider, butoanele cu colț tăiat, butonul de mute) și animațiile ușilor de airlock. Fiecare modul suprascrie doar variabilele de culoare-accent într-un `<style>` local, ca tot restul design-ului să rămână identic.